



FRONT-END JS

Clase n° 7 | Grid y Media Queries

¡Les damos la bienvenida! 🚀

👤 Vamos a comenzar a grabar la clase.

Índice

1

7. Grid y Media Queries

- ¿Qué es Grid?
- Implementación de Grid
- Maquetado con Flex y Grid
- Media Queries

2

8. Git y Github

- Git: Descarga de Git
- Crear un repositorio externo (Github)
- Comandos básicos (Init, commit, push)



¿Ya viste la "Introducción al programa" disponible en el campus virtual?



La visualización y resolución de un breve cuestionario es de carácter obligatorio para desbloquear los contenidos de las primeras 2 clases

Grid

Introducción a CSS Grid



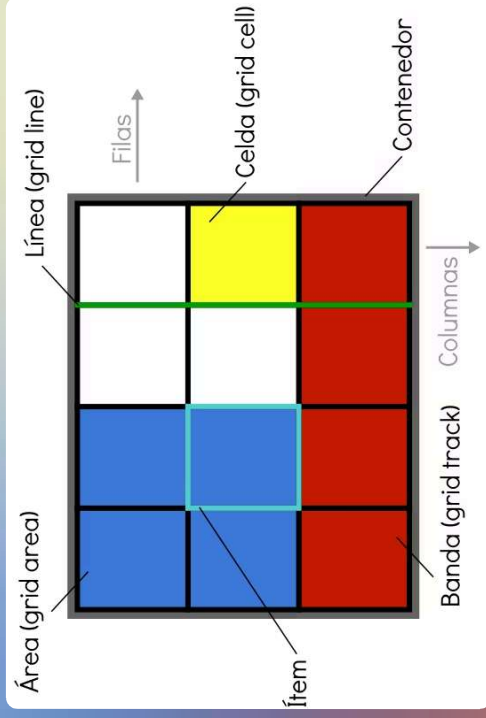
CSS Grid Layout

Permite organizar el contenido de una página web en una cuadrícula (grid) de filas y columnas.



Grid es bidimensional

Es posible acomodar el contenido en ambas direcciones al mismo tiempo. A diferencia de Flexbox, que solo te permite trabajar con una sola dimensión



Ventajas de CSS Grid



Control Total

Posicionamiento preciso de elementos en ambas dimensiones.



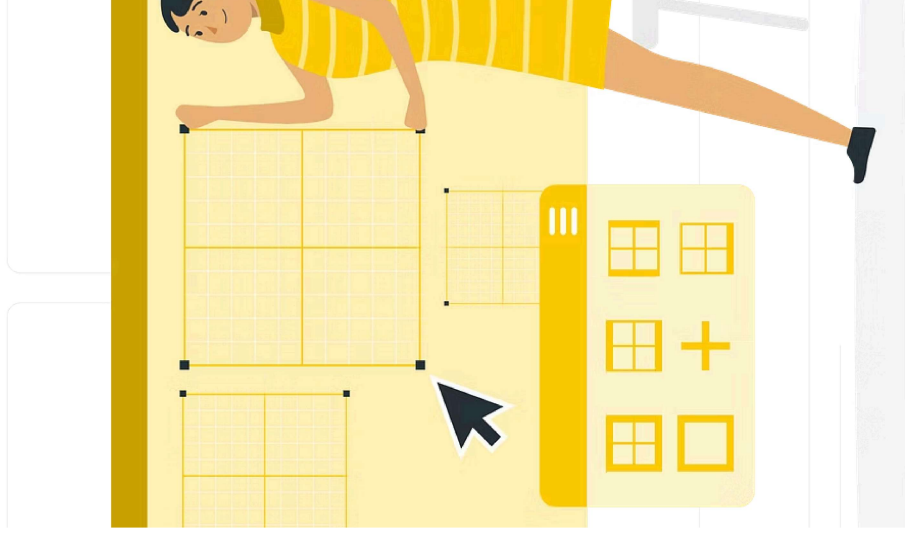
Diseños Complejos

Ideal para layouts con múltiples secciones y elementos.



Responsividad

Facilita la adaptación del diseño a diferentes tamaños de pantalla.



Implementación Básica de Grid

Contenedor Grid

Se define usando `display: grid` en el elemento contenedor.

Definición de Estructura

Se especifican columnas y filas con `grid-template-columns` y `grid-template-rows`.

Ejemplo básico de Grid:

```
<div class="grid-container">
  <div class="header">Cabecera</div>
  <div class="menu">Menú</div>
  <div class="main">Contenido Principal</div>
  <div class="footer">Pie de página</div>
</div>
```

CSS básico para Grid:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 200px 1fr; /* Una columna de 200px y otra que ocupa el espacio restante */  
  grid-template-rows: auto 1fr auto; /* Tres filas, la del medio es flexible */  
  gap: 10px; /* Espacio entre los elementos */  
}  
.header {  
  grid-column: 1 / 3; /* La cabecera ocupa las dos columnas */  
}  
.menu {  
  grid-column: 1 / 2; /* El menú ocupa la primera columna */  
}  
.main {  
  grid-column: 2 / 3; /* El contenido principal ocupa la segunda columna */  
}  
.footer {  
  grid-column: 1 / 3; /* El pie de página también ocupa las dos columnas */  
}
```

Propiedades Importantes de Grid

`grid-template-columns`

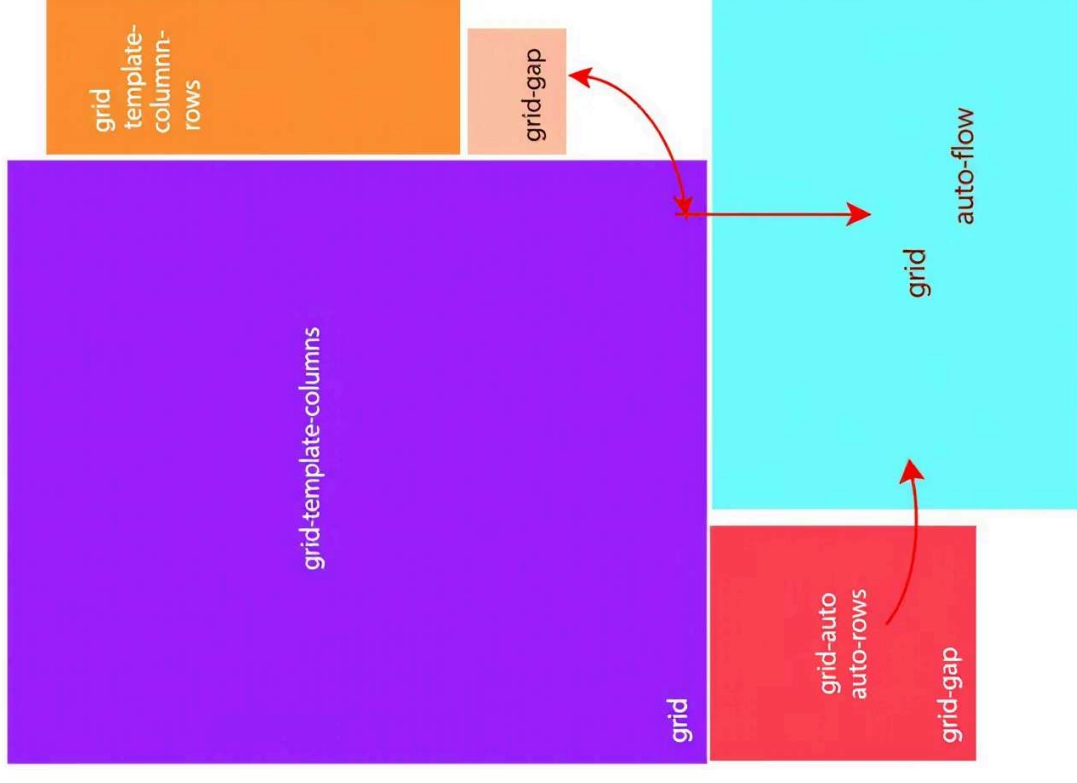
Define el número y tamaño de las columnas en el grid.

`grid-template-rows`

Especifica el número y tamaño de las filas en el grid.

`grid-column`

Controla en qué columnas se sitúa un elemento.



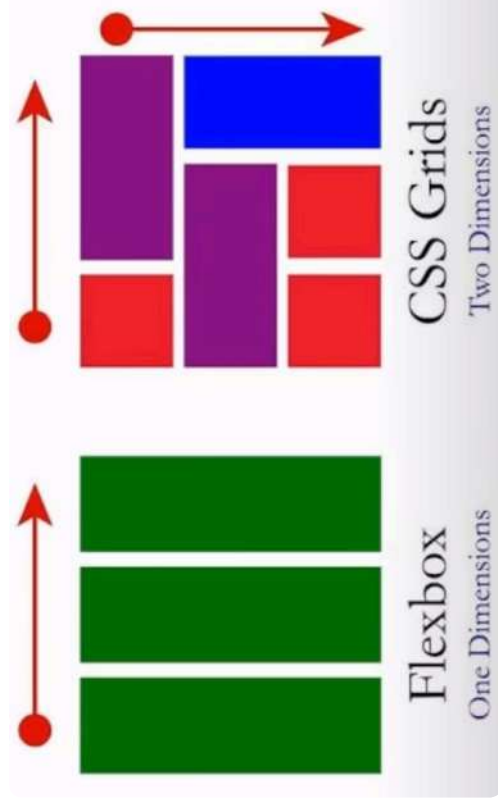
Maquetado con Flex y Grid

Grid para Estructura

Ideal para la estructura principal de la página, donde necesitas definir áreas específicas para cada sección

Flexbox para Componentes

Perfecto para alinear elementos dentro de secciones específicas. ya que te permite distribuir y alinear elementos a lo largo de una sola dimensión



Ejemplo de combinación

- Usamos Grid para organizar el layout principal de la página (cabecera, menú, contenido, pie de página).
- Dentro del menú (`.flex-menu`), utilizamos Flexbox para distribuir los elementos de la lista en una columna, con un espacio (`gap`) entre cada uno de ellos.

```
<div class="grid-container">
  <header class="header">Cabecera</header>
  <nav class="menu">
    <ul class="flex-menu">
      <li><a href="#">Inicio</a></li>
      <li><a href="#">Servicios</a></li>
      <li><a href="#">Contacto</a></li>
    </ul>
  </nav>
  <section class="main">Contenido principal</section>
  <footer class="footer">Pie de página</footer>
</div>
```

CSS:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: auto 1fr auto;  
  gap: 10px;  
}  
.flex-menu {  
  display: flex;  
  flex-direction: column;  
  gap: 5px;  
}  
.header, .footer {  
  grid-column: 1 / 3;  
}  
.main {  
  grid-column: 2 / 3;  
}
```

Conceptos Básicos de Grid



Grid Container

Es el contenedor que define la cuadrícula, configurado con `display: grid`.



Grid Items

Son los elementos dentro del grid container; se alinean en filas y columnas automáticamente



Grid Tracks

Espacio entre dos líneas adyacentes, ya sea una fila o una columna.



Grid Cell:

La intersección entre una fila y una columna, donde se coloca un grid item.



Grid Areas

Area rectangular dentro de la cuadrícula que puede estar ocupada por uno o varios grid items.

Propiedades Importantes en Grid

- **grid-template-columns:** define cuántas columnas tendrá tu grid y el tamaño de cada una.

```
.grid-container {  
  grid-template-columns: 1fr 2fr; /* Primera columna más pequeña, segunda más grande */  
}
```

- **grid-template-rows:** define el número de filas y su tamaño.

```
.grid-container {  
  grid-template-rows: auto 1fr; /* Primera fila automática, segunda ocupa el resto */  
}
```

- **grid-column / grid-row:** permite controlar cuántas filas o columnas ocupa un grid item.

```
.header {  
  grid-column: 1 / 3; /* Ocupa desde la columna 1 hasta la 3 */  
}
```

- **gap:** establece el espacio entre las filas y columnas del grid

```
.grid-container {  
  gap: 10px; /* Espacio uniforme entre filas y columnas */  
}
```

Media Queries

Introducción a Media Queries

Permiten aplicar estilos según características del dispositivo, como el ancho o la orientación de la pantalla.



Diseño Responsivo

Garantizan que el sitio web se adapte a diferentes tamaños, asegurando que se vea y funcione bien en todos los dispositivos.



Explicacion

La regla `@media` en CSS permite aplicar estilos condicionales según ciertas características del dispositivo:

- `@media only screen and (max-width: 600px)`: Estilos para pantallas de hasta 600px, como celulares.
- `@media only screen and (min-width: 768px)`: Estilos para pantallas de 768px o más, como tablets y laptops pequeñas.

Implementación de Media Queries, ejemplo:

```
/* Estilos generales */
body {
  font-size: 16px;
}

/* Estilos para pantallas pequeñas (menos de 600px) */
@media only screen and (max-width: 600px) {
  body {
    font-size: 14px;
  }
}

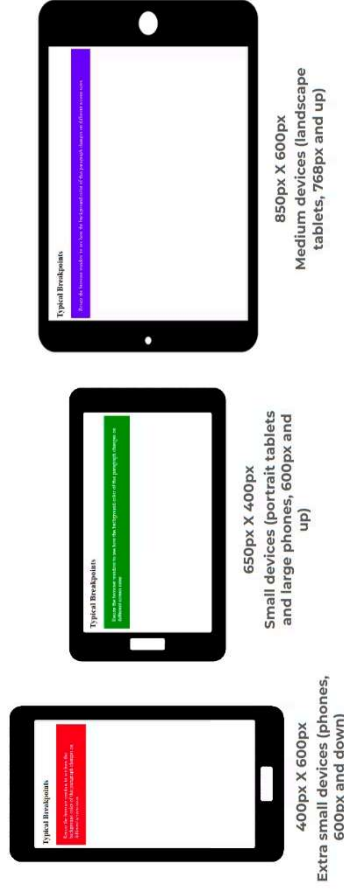
/* Estilos para pantallas medianas (más de 768px) */
@media only screen and (min-width: 768px) {
  body {
    font-size: 18px;
  }
}
```

Breakpoints en Media Queries

Define el ancho de pantalla en el que se aplican estilos específicos para garantizar que el diseño sea responsivo y adaptable.

Breakpoints comunes:

- **Hasta 600px:** Celulares y dispositivos pequeños.
- **600px - 768px:** Tablets en modo vertical.
- **768px - 992px:** Tablets en modo horizontal y laptops pequeñas.
- **992px - 1200px:** Laptops y monitores estándar.
- **Más de 1200px:** Monitores grandes.



Ejemplo de breakpoints:

```
/* Estilos para pantallas pequeñas */  
@media only screen and (max-width: 600px) {
```

```
.container {  
  grid-template-columns: 1fr; /* Una sola columna en pantallas pequeñas */  
}  
/* Estilos para pantallas medianas */  
@media only screen and (min-width: 768px) {  
  .container {  
    grid-template-columns: 1fr 1fr; /* Dos columnas en pantallas medianas */  
  }  
}  
/* Estilos para pantallas grandes */  
@media only screen and (min-width: 1200px) {  
  .container {  
    grid-template-columns: 1fr 1fr 1fr; /* Tres columnas en pantallas grandes */  
  }  
}
```



Ejercicios Prácticos



Ejercicio Práctico #1

Optativo | No entregable

Pasos a seguir:

1. Estructura HTML: Asegurate de tener una estructura básica que incluya una sección para "Reseñas" dentro de `<main>`, y que cada reseña esté dentro de un `<div>` con la clase `card`.
2. Aplicar CSS Grid: En el archivo `styles.css`, aplicá `display: grid` y `grid-template-columns` con `repeat(auto-fit, minmax(250px, 1fr))` para hacer que las cards sean

Sección "Reseñas"

Crear al menos 3 cards en la sección de "Reseñas", utilizando Grid, para organizar al menos tres cards de manera responsiva

responsivas y se adapten a la pantalla.

3. Márgenes y Espaciado: Usá gap para dar espacio entre las cards y padding para que no queden pegadas a los bordes del contenedor.
4. Interactividad: Añadí un efecto de hover con transform: translateY(-5px) para que las tarjetas se eleven al pasar el mouse.
5. Box Model: Aplicá padding, border y box-shadow para darle estructura a las cards y hacerlas más atractivas visualmente.



Ejercicio Práctico #1

Optativo | No entregable

Pasos a seguir:

- **Estructura HTML:** Crea una estructura básica con una sección "Contacto" y un formulario usando etiquetas como `<section>`, `<form>`, `<input>` y `<textarea>`.
- **Media Queries:** Usa `@media (max-width: 768px)` en `styles.css` para adaptar el diseño a dispositivos pequeños como tablets y celulares.
- **Ajuste de Cards:** Aplica `grid-template-columns: 1fr` a las tarjetas de reseñas para que se apilen en pantallas pequeñas y mejoren la legibilidad.

Media Queries

Implementar una media queries en la sección "Contacto" que ajuste el diseño de la página para dispositivos móviles, asegurándose de que el layout

de las cards de reseñas se adapte a pantallas más pequeñas.

- **Formulario Adaptable:** Modifica el padding y los tamaños de los campos del formulario para optimizar su apariencia en dispositivos con menos espacio.
- **Pruebas:** Verifica el diseño en herramientas de desarrollador para confirmar que las media queries funcionan y el contenido se adapta a diferentes tamaños de pantalla.